

BigCodeBench/797 Evaluations

Task Description

Count the total number of brackets (i.e., '(', ')', '{', '}', '[', ']') in a pandas DataFrame.

- Pattern to use: `'[(){}[\]]'`
 - Must raise `TypeError` if input is not a `DataFrame`
 - Output: `int` (total bracket count)
 - Example: `df` with `['(a)', 'b', '[[[]])c']` should return 8
-

Model 1 Evaluation

model1_efficiency

Total steps: 7

- Step 1: Problem understanding - **Essential**
- Step 2: Input/output specification - **Essential**
- Step 3: Constraints and edge cases - **Essential**
- Step 4: Design approach - **Essential**
- Step 5: Strategy refinement - **Essential**
- Step 6: Example walkthrough with validation - **Essential** (though verbose)
- Step 7: Final code structure - **Essential**

model1_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - clarifies I/O
- Step 3: Coherent - identifies constraints
- Step 4: Coherent - design decisions follow from constraints
- Step 5: Coherent - refines approach
- Step 6: Coherent - validates approach with example
- Step 7: Coherent - concludes with implementation

Internal Consistency: No contradictions. Correctly identifies and corrects initial counting error in example.

model1_completeness

Problem Coverage:

- Type validation: Yes
- Count all brackets: Yes
- Use specified pattern: Yes
- Handle non-string data: Yes
- Return integer: Yes

Factual Correctness: All correct, including regex pattern interpretation

Constraint and edge case handling:

- Empty DataFrame
 - Non-string elements
 - NaN values
 - Very comprehensive
-

Model 2 Evaluation

model2_efficiency

Total steps: 5

- Step 1: Input validation - **Essential**
- Step 2: Define regex pattern - **Essential**
- Step 3: Iterate over cells - **Essential**
- Step 4: Count matches - **Essential**
- Step 5: Return result - **Non-essential** (already implied)

model2_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - pattern definition
- Step 3: Coherent - iteration strategy
- Step 4: Coherent - counting method
- Step 5: Coherent - final return

Internal Consistency: No contradictions, but less detailed than Model 1

model2_completeness

Problem Coverage:

- Type validation: Yes
- Count all brackets: Yes
- Use specified pattern: Yes (mentions correct escaping)
- Handle non-string data: Yes (converts to string)
- Return integer: Yes

Factual Correctness: Correct understanding of regex escaping

Constraint and edge case handling: Basic coverage, less comprehensive than Model 1

Model 3 Evaluation

model3_efficiency

Total steps: 8

- Step 1: Type validation - **Essential**
- Step 2: Initialize counter - **Essential**
- Step 3: Store pattern - **Essential**
- Step 4: Access all elements - **Essential**
- Step 5: Type check for strings - **Essential**
- Step 6: Find matches - **Essential**
- Step 7: Count and accumulate - **Essential**
- Step 8: Return result - **Essential**

model3_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - setup
- Step 3: Coherent - pattern storage
- Step 4: Coherent - data access strategy
- Step 5: Coherent - type filtering
- Step 6: Coherent - pattern matching
- Step 7: Coherent - counting

- Step 8: Coherent - return

Internal Consistency: Consistent throughout, clear linear progression

model3_completeness

Problem Coverage:

- Type validation: Yes
- Count all brackets: Yes
- Use specified pattern: Yes
- Handle non-string data: Yes (type check)
- Return integer: Yes

Factual Correctness: All statements correct

Constraint and edge case handling:

- Non-string values
 - NaN handling mentioned
 - Good coverage
-

Model 4 Evaluation

model4_efficiency

Total steps: Extended reasoning (not clearly delineated)

- Problem analysis - **Essential**
- Multiple approach considerations - **Non-essential** (excessive exploration)
- Pattern analysis (very detailed) - **Partially non-essential** (too verbose)
- Implementation options - **Non-essential** (multiple redundant approaches)
- Final implementation - **Essential**

model4_logic

Step Coherence:

- Explores multiple approaches (iterative, vectorized, compiled regex)
- Sometimes circular reasoning when discussing pattern
- Eventually arrives at correct solution

Internal Consistency: Some redundancy and exploration of alternatives that could be more focused

model4_completeness

Problem Coverage:

- Type validation: Yes
- Count all brackets: Yes
- Use specified pattern: Yes (extensive discussion)
- Handle non-string data: Yes
- Return integer: Yes

Factual Correctness: Correct, though overly detailed regex analysis

Constraint and edge case handling:

- Empty DataFrame
 - Multiple columns
 - NaN values
 - Performance considerations
 - Very thorough but verbose
-

Model 5 Evaluation

model5_efficiency

Total steps: 4

- Step 1: Input validation - **Essential**
- Step 2: Convert and combine data - **Essential**
- Step 3: Apply regex pattern - **Essential**
- Step 4: Count and return - **Essential**

model5_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - data preparation
- Step 3: Coherent - pattern matching
- Step 4: Coherent - final count

Internal Consistency: No contradictions, very concise

model5_completeness

Problem Coverage:

- Type validation: Yes
- Count all brackets: Yes
- Use specified pattern: Yes
- Handle non-string data: Yes (astype(str))
- Return integer: Yes

Factual Correctness: Correct approach

Constraint and edge case handling: Minimal discussion of edge cases

Model 6 Evaluation

model6_efficiency

Total steps: 5

- Step 1: Problem understanding - **Essential**
- Step 2: Example breakdown - **Essential**
- Step 3: Solution approach - **Essential**
- Step 4: Edge case considerations - **Essential**
- Step 5: Implementation strategy - **Essential**

model6_logic

Step Coherence:

- Step 1: Starting point
- Step 2: Coherent - validates understanding with example
- Step 3: Coherent - outlines solution steps
- Step 4: Coherent - identifies edge cases
- Step 5: Coherent - implementation approach

Internal Consistency: No contradictions, clear progression

model6_completeness

Problem Coverage:

- Type validation: Yes
- Count all brackets: Yes
- Use specified pattern: Yes

- Handle non-string data: Yes
- Return integer: Yes

Factual Correctness: Correctly breaks down example ($2+0+6=8$)

Constraint and edge case handling:

- Non-string elements
 - NaN/None values
 - Empty DataFrames
 - Good coverage
-

Summary

- **Most Complete:** Models 1 and 4 (very thorough edge case handling)
- **Most Efficient:** Model 5 (only 4 essential steps)
- **Best Logic:** Models 1 and 6 (clear, coherent progression)
- **Verbose/Inefficient:** Model 4 (excessive exploration of alternatives)